

Programming for Essential Digital Skills, Chapter 10

Christoph Walsh

2026

Univariate Unconstrained Optimization

- Suppose we want to find the x that minimizes the following function:

$$f(x) = -8 - 2x + x^2$$

- Using analytical methods, we take the first derivative:

$$f'(x) = -2 + 2x$$

- We then find the value x that solves $f'(x) = 0$. This is $x = 1$.
- We check the sign of the second derivative to see if this is a maximum or minimum:

$$f''(x) = 2$$

- This is positive for all values of x , so $x = 1$ is a minimum of this function.

Using Python

- Sometimes we need to optimize a function where analytical methods cannot be used, or are difficult to use.
- We can use Python to numerically optimize a function in these cases.
- We learn how to do this 2 ways:
 - 1 Plotting approach
 - 2 Using `scipy.optimize`.

Plotting Approach

- One way to find the minimum point is to plot the function and to see visually where the minimum is.
- To do this we need to first generate values of x and $f(x)$ to make the plot.
- For the values of x , we want many values between a lower and upper bound that we choose.
- Knowing what bounds to use is often not known in advance, but it can be chosen using trial and error.

Generating Values of x

- For our example function, we will use the bounds -4 and $+6$.
- We generate 20 values of x equally-spaced between -4 and $+6$ (range is 10):

```
x = [-4 + i * (10/19) for i in range(20)]
```

- When $i = 0$, the first value, we get -4 .
- When $i = 1$, we get $-4 + \frac{10}{19}$.
- When $i = 19$, the last value, we get 5 .

Generating Values of x

- The function `linspace` in `numpy` does exactly this in an easier way:

```
import numpy as np
x = np.linspace(-4, 6, 20)
```

Generating Values of $f(x)$

- For all these values of x we calculate the corresponding output of the function, which we will call y .
- We define the function f and use a list comprehension:

```
def f(x):  
    return x**2 - 2*x - 8  
  
y = [f(i) for i in x]
```

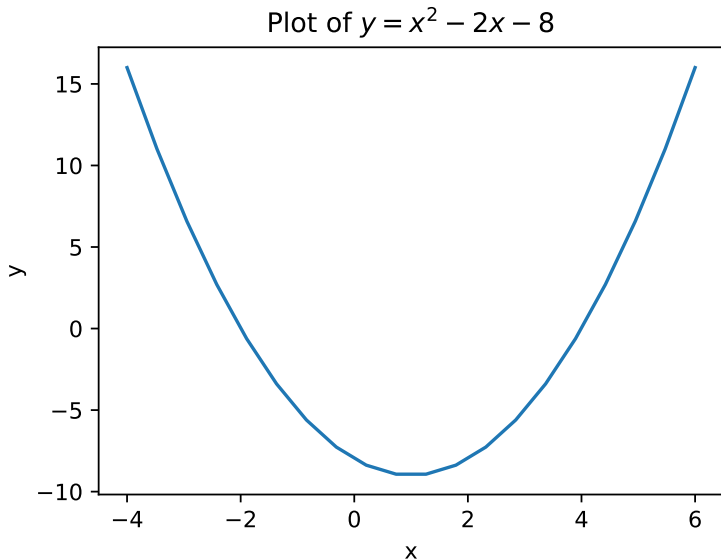
Plotting x and $f(x)$

- Now with this x and y we can make the plot as follows:

```
import matplotlib.pyplot as plt
plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Plot of  $y=x^2-2x-8$ ")
```

(Output on next slide)

Plotting x and $f(x)$



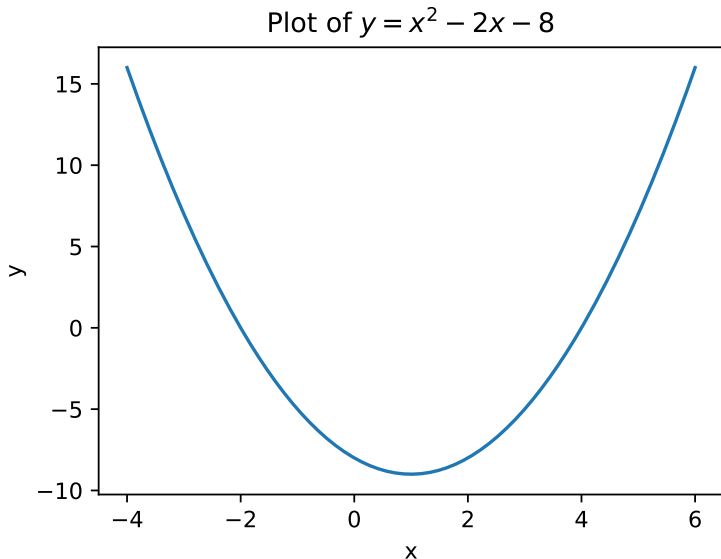
Improving the Plot Resolution

- If you look closely at the plot you can see that it is not very smooth and has some kinks in it.
- This is because we used only 20 values to make it.
- Let's do it again with 200 values:

```
x = np.linspace(-4, 6, 200)
y = [f(i) for i in x]
plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Plot of  $y=x^2-2x-8$ ")
```

(Output on next slide)

Improving the Plot Resolution



Using Optimization

- We will now use Python to find the extreme point using optimization.
- We can do this using the `minimize()` function from the module `scipy.optimize`.
- When using this function we need to provide an initial guess of where the minimum is.

```
from scipy.optimize import minimize
result = minimize(f, x0=0)
result['x']
```

```
array([1.00000001])
```

Local vs Global Minima

- Why do we need to give an initial guess?
- Sometimes a function can have several local minima.
- A univariate function has a local minimum at x if $f(x + \epsilon) > f(x)$ and $f(x - \epsilon) > f(x)$ for small $\epsilon > 0$.
- But there could be another value x' where $f(x') < f(x)$.
- A function has a global minimum at x if $f(x') > f(x)$ for all $x' \neq x$.
- If a function has multiple local minima, the minimize function is not guaranteed to find the global minimum.
- Given an initial guess it searches for the minimum according to an algorithm starting at your initial guess.
- If you start the algorithm elsewhere, it may find another minimum.

Local vs Global Minima

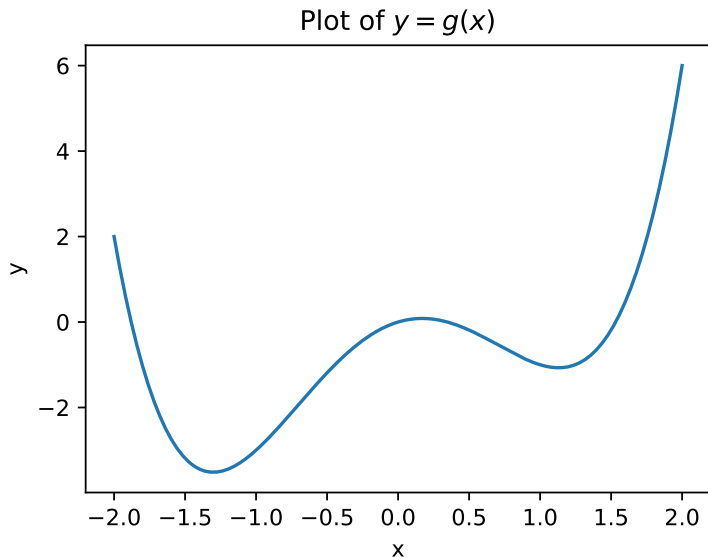
- Let's plot the following function which shows an example of this:

$$g(x) = x^4 - 3x^2 + x$$

```
def g(x):  
    return x**4 - 3*x**2 + x  
x = np.linspace(-2, 2, 200)  
y = [g(i) for i in x]  
plt.plot(x, y)  
plt.xlabel("x")  
plt.ylabel("y")  
plt.title("Plot of $y=g(x)$")
```

(Output on next slide)

Local vs Global Minima



Local vs Global Minima

- We can see that there are two local minima: one near -1.2 (the global minimum) and one near $+1.2$ (a local minimum).
- If we use the `minimize()` function starting at 1, the algorithm stops at the local minimum:

```
result1 = minimize(g, x0=1)
result1['x']
```

```
array([1.13090111])
```

- If we start it at -1 , we find the global one:

```
result2 = minimize(g, x0=-1)
result2['x']
```

```
array([-1.30083962])
```

Local vs Global Minima

- Bad choices of starting values could lead to failing to find the global minimum.
- Therefore it is good practice to perform minimize from several different starting values.
- We can repeat it 20 times for starting values in the range -10 and $+10$ and only keep the best one:

```
initial_guesses = np.linspace(-10, 10, 20)
best_result = None

for x0 in initial_guesses:
    result = minimize(g, x0=x0)

    if best_result is None or result.fun < best_result.fun:
        best_result = result
best_result['x']

array([-1.30083957])
```

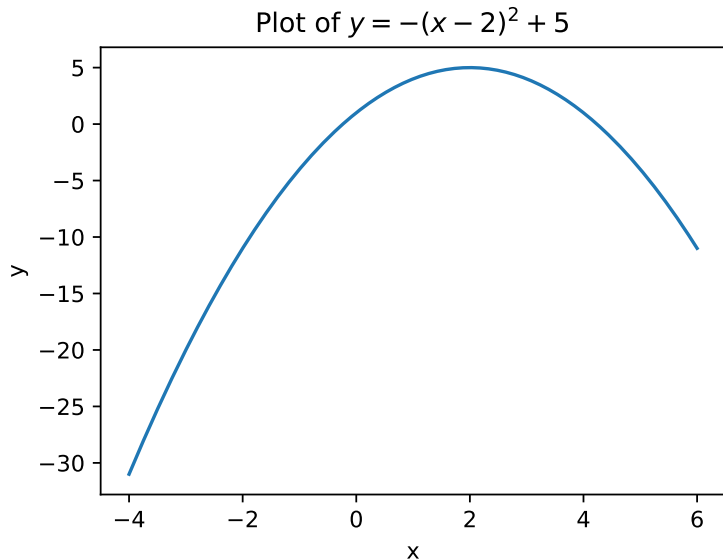
Maximizing a Function

- If instead of minimizing a function you want to maximize it instead, you will still use the minimize function from `scipy.optimize`.
- To do this you minimize the negative of the output of the function, which is the same as maximizing it.
- Making the output of a function negative flips it upside down and turns a maximization problem into a minimization problem.

```
def h(x):  
    return -(x - 2)**2 + 5  
x = np.linspace(-4, 6, 200)  
y = [h(i) for i in x]  
plt.plot(x, y)  
plt.xlabel("x")  
plt.ylabel("y")  
plt.title("Plot of  $y=-(x-2)^2 + 5$ ")
```

(Output on next slide)

Maximizing a Function



Maximizing a Function

To find the maximum, we use a `lambda` function to turn the output negative and find the minimum:

```
result = minimize(lambda x: -h(x), x0=0)
result['x']
```

```
array([2.00000002])
```

- Note that the `result` shows the function value at the optimum being -5 , whereas the plot of the function shows it at $+5$.